통합반 6회차 실습 1 — 업무도구연결호출

자료 다운로드와 작업 공간

앞 에이전트에 필요한 읽기 도구를 연결하고, 실제 호출의 입력과 근거가 맞는지 확인하겠습니다. 실습 자료를 받아 압축을 풀고 VS Code에서 작업 공간을 열겠습니다.

기본 등록 도구와 학생 서버 연결

포털의 도구 MCP 메뉴에서 기본 업무 DB와 판단 엔진 서버를 열겠습니다. 이름, 연결 방식, 검사 상태와 도구 목록을 보겠습니다. 읽기 표시가 있는 조회 도구 하나를 골라 이름과 설명, 필수 입력을 적습니다. 판단 제출처럼 쓰기인 도구는 이번 조회 시험의 대상에서 제외합니다.

이제 제공받은 읽기 서버를 제 이름으로 등록하겠습니다. HTTP 서버라면 포털 실행 환경에서 닿을 주소와 연결 방식을 입력합니다. 제 PC의 로컬 경로를 포털 서버에서도 쓸 수 있다고 가정하지 않겠습니다. 인증이 필요하면 포털이 제공하는 비밀값 설정에 입력하고 설정 파일에는 실제 값을 남기지 않습니다.

연결 검사를 누르고 도구 목록이 돌아왔는지 보겠습니다. 실패하면 표시된 분류에 따라 주소·서버 상태·인증·실행 위치를 확인합니다. 설정을 고친 뒤에는 검사를 다시 합니다. 읽기 표시와 허용된 도구 이름을 대조하겠습니다. 목록에 보이는 것과 에이전트가 사용할 수 있는 것은 다릅니다.

읽기 호출과 기존 에이전트 연결

허용된 조회 도구의 입력 스키마를 보며 수업 대상의 실제 식별자를 채우겠습니다. 입력과 결과의 대상을 대조하고 호출을 요청합니다. 반환된 행과 기준 시각, 호출 기록의 서버·도구·입력을 나란히 확인하겠습니다. 정상적으로 0행이면 해당 조건의 행을 못 찾은 것이고, 실패나 시간 초과라면 값을 확인하지 못한 것입니다. 서로 바꾸어 적지 않겠습니다.

앞 회차에 만든 에이전트를 열고 검사한 서버와 필요한 읽기 도구를 연결하겠습니다. 기존 도구와 지침을 보존한 채 이번 조회에 필요한 항목만 더합니다. 같은 업무 요청을 한 번 전달한 뒤 실제로 사용한 도구와 입력, 반환 값을 호출 기록에서 확인합니다. 도구를 쓰지 않았다면 성공으로 기록하지 않고 요청과 지침, 연결·허용 목록을 살펴보겠습니다.

저장과 다시 확인

검사 시각, 허용 도구, 조회 대상과 반환 행, 에이전트 연결 변경점과 호출 기록을 저장하겠습니다. 파일을 다시 열어 설정 검사와 업무 호출의 결과를 구분해 읽습니다. 이제 로컬 작업 공간에서 세 서버를 함께 연결하겠습니다.

통합반 6회차 실습 2 — MCP서버조합

다운로드와 작업 공간 열기

쿨러 점검 자료와 변경 이력, 기준 시각을 서로 다른 도구에서 읽어 하나의 확인 보고서로 연결하겠습니다. 실습 자료를 받아 압축을 풀고 VS Code에서 작업 공간을 열겠습니다.

로컬 작업 자료와 세 서버 설정

코드/도구의 자료_준비.py를 열어 어떤 폴더와 Git 이력을 만드는지 읽겠습니다. 수업에서 코드 폴더의 터미널에 python 도구/자료_준비.py를 실행하면 이 실습 폴더 안에 업무자료가 준비됩니다. 이미 업무자료가 있으면 그대로 두는 기본 실행만 사용하겠습니다. 다시 만들기 옵션은 쓰지 않습니다. 업무자료와 제공된 구매팀_자료는 연습용 자료이며 실제 업무 저장소가 아닙니다.

VS Code 작업 공간의 .vscode/mcp.json을 만들겠습니다. 최상위 servers 아래 filesystem, git, time을 각각 둡니다. 각 항목의 type은 stdio, command는 교육 환경의 Python 실행 파일로 정하고 args에 해당 서버 스크립트의 절대경로를 넣습니다. 파일 서버에는 --read-only와 업무자료 절대경로, Git 서버에는 --repository와 같은 업무자료 경로를 추가합니다. 시간 서버에는 서버/시간_서버.py 경로를 넣습니다. 제공 설정_작성.md의 형식과 실제 경로를 대조하며 적겠습니다.

이 로컬 설정은 이론의 HYD mcpServers 설정을 그대로 붙여 넣는 것이 아닙니다. VS Code는 이 파일의 servers를 읽습니다. HYD가 서버에서 실행된다면 강사 PC의 로컬 경로가 그 서버에서도 보이는 것은 아니므로 포털에 로컬 경로를 등록하지 않겠습니다.

연결하고 한 요청으로 조합하기

세 서버를 각각 시작하고 도구 목록을 확인하겠습니다. 파일 서버의 허용 폴더와 읽기 전용 모드, Git 서버의 대상 저장소, 시간 서버의 시간대 입력을 기록합니다. 실패하면 실행 파일·스크립트 경로·대상 폴더·서버 오류를 순서대로 확인합니다. 한 서버가 실패했다고 다른 두 서버까지 성공으로 묶지 않겠습니다.

이제 '업무자료에서 쿨러 점검 메모를 읽고, Git에서 최근 변경과 아직 커밋하지 않은 변경을 확인한 뒤, 시간 도구의 Asia/Seoul 현재 시각을 기준으로 남은 확인 사항을 정리해 주세요. 파일 경로·커밋·조회 시각을 각 근거에 남기고, 과거 회의록의 상대 날짜는 회의 날짜를 기준으로 해석해 주세요'라고 요청하겠습니다. 세 도구가 실제로 호출됐는지 기록을 펼쳐 보고 output/combined.md의 각 문장을 원문과 이력으로 대조합니다. 현재 시간과 문서 작성 날짜를 섞어 옛 기한을 새 기한으로 바꾸지 않겠습니다.

실패와 권한 경계 확인

연습용 구매팀_자료는 파일 서버 허용 범위 밖에 두었습니다. 그 파일을 요청했을 때 접근이 거부되고 미확인으로 남는지 보겠습니다. 범위를 몰래 넓히는 대신 이 요청에서 필요하지 않은 자료라고 기록합니다. 결과가 일부만 보이면 읽은 범위를 적고 필요한 구간을 다시 확인합니다. 조회 실패나 시간 초과는 값 없음이나 업무 완료로 바꾸지 않겠습니다.

저장하고 재확인

설정 파일, 서버별 목록과 호출 기록, 결합 결과, 오류와 수정 내역을 저장하겠습니다. 다시 열어 결과의 파일 근거와 Git 변경, 조회 시각이 서로 맞는지 봅니다. 서버 연결 성공과 답변의 정확성은 별개입니다. 다음에는 이렇게 확인한 도구와 별개로, 반복해서 쓸 업무 지침을 직접 작성하겠습니다.

통합반 6회차 실습 3 — 회의록스킬작성

다운로드와 스킬 폴더 만들기

설비보전 회의에서 결정한 일과 아직 남은 일을 구분하는 스킬을 만들겠습니다. 다음 회의에도 같은 기준이 적용돼야 합니다. 실습 자료를 받아 압축을 풀고 VS Code에서 작업 공간을 열겠습니다.

이번에는 프로그램 내부보다 반복해서 사용할 업무 지침이 대상입니다. 두 회의록에 어떤 결정과 미결 사항이 담겼는지 먼저 확인하겠습니다.

코드 폴더를 작업 공간으로 열고 .github/skills/meeting-actions 폴더에 SKILL.md를 새로 만들겠습니다. 머리말의 name은 meeting-actions, description은 '회의록에서 결정사항과 후속 작업·담당자·기한을 근거와 함께 정리할 때 사용한다'로 적겠습니다. 폴더 이름과 name을 맞추고, 도우미가 실제로 이 스킬을 인식하는지 확인합니다. 파일이 있다는 사실만으로 적용됐다고 말하지 않겠습니다.

절차와 예외 직접 작성하기

본문에는 입력, 수행 순서, 예외, 출력을 적겠습니다. 입력은 회의록 원문이고, 순서는 원문 구간을 읽고 결정과 제안·미결정을 구별한 뒤 후속 작업을 정리하는 것입니다. 작업마다 내용·담당자·기한·상태·근거 문장을 남기게 하겠습니다. 담당자나 기한이 없으면 미정으로 표시하고, 여러 사람이 적혀 있으면 한 명을 임의로 책임자로 고르지 않도록 적습니다.

'원인이 금형 쪽이면 점검한다'는 조건부 작업이지 이미 확정된 작업이 아닙니다. '정하지 않고 넘어갔다'는 제안을 결정으로 바꾸지 않도록 예외를 적겠습니다. '내일까지'는 회의 날짜가 분명할 때만 날짜로 풀고 원문 표현과 해석 기준을 함께 남기겠습니다. '이번 주 안'처럼 기준이 여럿일 수 있는 말은 임의 마감 시각을 만들지 않겠습니다.

출력은 output/actions.md와 output/actions.json으로 정하겠습니다. 같은 항목 ID를 써서 두 형식이 일치하도록 하고, 원문 위치와 확인이 필요한 항목을 포함하겠습니다. 이 파일은 일하는 방법을 적은 지침입니다. 일정 등록이나 알림 발송 권한이 생기는 것은 아닙니다.

두 회의록으로 적용과 원문 대조

도우미에 meeting-actions 스킬로 첫 회의록을 정리해 달라고 요청하겠습니다. 적용 기록을 확인한 뒤 결과와 원문을 나란히 엽니다. 결정과 제안이 섞이지 않았는지, 담당자와 기한을 지어내지 않았는지, 이미 끝난 일을 새 할 일로 만들지 않았는지 확인하겠습니다. 틀린 항목은 결과만 고치는 데서 멈추지 않고 SKILL.md의 해당 규칙을 고쳐 다시 적용합니다.

이번에는 두 번째 회의록을 같은 스킬로 처리하겠습니다. 이 자료에는 담당자가 없는 확인 사항, 두 사람이 맡는 현장 안내, 조건부 점검과 보류된 양식 변경이 있습니다. 이 차이가 결과에 그대로 남는지 보겠습니다. 첫 회의록의 이름과 날짜가 새 결과에 섞이면 입력과 재사용 지침을 분리해 고치겠습니다.

회의 사이에서 이어진 조치 확인

두 결과를 요청별로 다시 대조하겠습니다. 정기 회의의 구매팀 전달과 주간 협의의 전달 완료·견적 확인은 이어지는 기록입니다. 문장이 비슷하다고 하나를 지우거나, 같은 일을 새 작업으로 두 번 만들지 않겠습니다. 이전 결정, 후속 확인, 아직 남은 일을 각각 남기도록 스킬을 보완합니다.

이번 수정이 새 회의록에만 맞춘 예외가 아닌지 보겠습니다. 이름이나 특정 날짜를 지침에 고정하지 않고, 출처와 상태 변경을 확인하는 규칙으로 남기겠습니다.

저장과 재확인

SKILL.md와 두 회의록별 출력, 수정 이유를 저장하겠습니다. 새 입력에 같은 방법을 썼는지, 한 번의 결과를 복사한 것은 아닌지 다시 확인합니다. Markdown과 JSON의 작업 수·ID·담당자·기한도 대조합니다. 이어서 같은 출처 확인 원칙을 학습 코치에 적용하겠습니다.

통합반 6회차 실습 4 — 학습코치

다운로드와 학습 자료 열기

앞에서 정리한 개념과 출처로 답의 이유를 확인해 주는 학습 코치를 만들겠습니다. 실습 자료를 받아 압축을 풀고 VS Code에서 작업 공간을 열겠습니다.

대화 지침 직접 만들기

coach.md를 새로 만들고 목적을 '제공 자료를 근거로 한 문제씩 묻고 답의 이유를 설명하는 학습 코치'로 적겠습니다. 사용할 파일과 절을 지정하고, 문항을 한 번에 하나만 제시한 뒤 답변을 기다리도록 적습니다. 답변 전에는 정답을 공개하지 않고, 답변 뒤에는 판정·정답·이유·자료의 절과 근거 문장을 함께 주도록 하겠습니다.

자료만으로 답이 정해지지 않으면 문항을 내지 않거나 판단 불가로 남깁니다. 학생이 틀렸다고 해서 자료에 없는 설명을 만들어 보충하지 않도록 적겠습니다. 답변을 받은 뒤에만 다음 문항으로 넘어가고, 종료 요청을 받으면 진행한 문항과 다시 볼 절을 정리하도록 하겠습니다.

문항과 피드백 확인

GitHub Copilot 채팅에 coach.md와 입력 자료를 지정해 학습을 시작하겠습니다. 첫 문항이 자료의 어느 내용에 근거하는지 보되 정답을 미리 노출하지 않는지 확인합니다. 강사가 학습자 역할로 답을 직접 입력하겠습니다. 이 답변은 실제 학습자의 성적이 아닌 기능 확인용 입력입니다.

피드백이 돌아오면 원문과 대조합니다. 정답 여부와 이유가 일치하는지, 적힌 절에 근거가 있는지 보겠습니다. 이어지는 문항도 현재 이해한 내용으로 직접 답하겠습니다. 맞거나 틀렸다는 판정은 실제 답변과 원문으로 확인하고, 출처 문장이 그 판단을 뒷받침하는지 보겠습니다. 틀린 답이나 근거 부족이 실제로 확인되면 교정 또는 판단 보류의 이유를 읽습니다. 이번에 나오지 않은 경우는 미확인으로 남기고, 오답 장면을 만들기 위해 답을 바꾸지 않겠습니다.

문항을 한꺼번에 여러 개 내거나 답변 전에 정답을 공개하면 coach.md의 순서 규칙을 고칩니다. 새 대화에서 같은 자료로 다시 확인하고, 잘못된 피드백은 이전 기록에 수정 이유를 남겨 구분하겠습니다. 텍스트 파일로 이 흐름을 완주하며 Notion API 연결은 필요하지 않습니다.

저장과 마무리

입력 자료 목록, coach.md, 대화 기록과 원문 대조를 output에 저장하겠습니다. 다시 열어 문항과 내 답, 판정, 정답과 출처, 다음 문항이 순서대로 남았는지 확인합니다. 자료를 읽지 못했거나 판단이 남은 부분은 review.md에 적고, 이번 회차의 도구와 지침을 함께 정리하겠습니다.